

Philosophical Bias in Pre-Trained Language Models: A Comparative Study of Eastern and Western Training Corpora

Vanshika Singla, Mahatav Arora, Bowen Hui

May 1, 2026

Abstract

1 Introduction

Large language models acquire knowledge from their training data. The text a model encounters during pre-training determines its vocabulary, conceptual scope, and default responses in uncertain situations. Two models with identical architectures and training procedures will generate different outputs if their training corpora originate from distinct traditions, and these differences may not always be reflected in perplexity scores.

In this thesis, we train two GPT-2 models from scratch: one on a corpus of Western philosophy and the other on a corpus of Eastern philosophy. Both models share the same architecture, tokenizer, and hyperparameters; the only variable is the training text. Therefore, any measurable differences in their outputs that cannot be attributed to chance must originate from the corpus.

We halt the process at the end of pre-training, without fine-tuning or any post-training. This approach is chosen because instruction-tuned models reflect the influence of instruction authors as much as their original corpora, making it difficult to attribute specific behaviours to either stage. By stopping after pre-training, we can isolate the effects of the corpus itself.

The primary challenge lies in measurement. Since the two models are trained on disjoint corpora, there is no held-out test set that is equally representative for both, and same-model perplexity is not directly comparable. The main metric employed is cross-perplexity: each model evaluates text generated by the other, and high cross-perplexity values in both directions indicate that the models have learned distinct distributions. Additional metrics include concept-marker frequencies (assessing whether each model utilizes the technical vocabulary of its respective tradition), type-token ratio and repetition (evaluating output fluency and potential model collapse), and the Bhattacharyya overlap of the output distributions. Some metrics yield inconsistent results; Section 4 discusses the rationale for considering both outcomes valid.

Section 2 provides background information. Section 3 details the corpora, the GPT-2 configuration, and the progressive training pipeline. Section 4 presents the results, and Section 5 discusses the successes and limitations of the approach.

2 Background

2.1 Mathematical Definition: Manu

A language model is a system that, given some text, predicts what text is likely to come next. This section makes that idea precise by expressing it in the language of probability, which is the form we will need to train and evaluate one.

2.1.1 Problem in Mathematical Terms: Manu

We frame language modelling as a *next-token prediction* task [BDVJ03, Gol16]. Rather than asking “what is the next sentence?”, we ask “what is the next small piece of text given everything that has

come before?” Answering the small question repeatedly turns out to be enough to answer the larger one.

A token is a small unit of text drawn from a fixed list called the *vocabulary* [JM23]. Depending on the system, a token might be a whole word, a subword piece (*run*, *##ning*), or even a single character. The vocabulary itself is finite and decided in advance, and we write it as

$$V = \{w_1, w_2, \dots, w_{|V|}\}, \quad (1)$$

where $|V|$ is the number of tokens in it.

A piece of text is then a *sequence* of tokens. For a sequence of length T , we write

$$\mathbf{x} = (x_1, x_2, \dots, x_T), \quad (2)$$

with each x_i drawn from the vocabulary,

$$x_i \in V \quad \text{for } i = 1, \dots, T. \quad (3)$$

A language model assigns a probability to any such sequence: how likely is this particular ordering of tokens to appear in real text? We write this joint probability as

$$P(x_1, x_2, \dots, x_T). \quad (4)$$

Working directly with the joint probability of an entire sequence is awkward because the number of possible sequences is enormous (Section 2.1.2 makes this concrete). The *chain rule of probability* gives us a way around it by rewriting the joint probability as a product of simpler conditional probabilities:

$$P(x_1, x_2, \dots, x_T) = \prod_{i=1}^T P(x_i \mid x_1, x_2, \dots, x_{i-1}). \quad (5)$$

So modelling whole sequences and modelling the next token given the previous ones are the same problem, repeated T times. From here on, we only need a way to compute $P(x_i \mid x_1, \dots, x_{i-1})$.

2.1.2 Why a Direct Lookup Table Doesn’t Work: Manu

The most obvious way to compute these conditional probabilities is to memorize them: store, for every possible context, the probability of every possible next token in a giant table, and look up the answer whenever we need one. This approach fails for two reasons. The table would be too large to store, and even if we could, almost all its entries would be empty.

Suppose our tokens are English words and our vocabulary contains roughly 50,000 of them:

$$|V| = 50,000 \quad (6)$$

$$\mathbf{x} = (x_1 = \text{hello}, x_2 = \text{phone}, \dots, x_T). \quad (7)$$

The number of possible sequences of length T is

$$|V|^T, \quad (8)$$

since each of the T positions can be filled by any of the $|V|$ tokens. For a fairly short sequence of $T = 20$ tokens, this gives

$$50,000^{20} \quad (9)$$

possible sentences. At 8 bytes per stored probability, the table would need around $8 \times 50,000^{20} \approx 8 \times 10^{94}$ bytes, which is far beyond any practical storage capacity.

The sparsity problem is the second issue. A natural way to estimate a probability is to count how often something happens and divide by the total number of observations. For sequences this gives the maximum-likelihood estimate

$$P(x_1, x_2, \dots, x_T) = \frac{\text{count}(x_1, x_2, \dots, x_T)}{N}, \quad (10)$$

where N is the total number of sequences in our dataset. Even with a very large dataset, say $N = 10^{12}$ sequences, the vast majority of length-20 sequences will never have appeared in it. For all of those unseen sequences,

$$P(x_1, x_2, \dots, x_T) = \frac{0}{10^{12}} = 0. \quad (11)$$

The table, therefore, assigns probability zero to almost every sentence a person might actually type, including ordinary English sentences that happen to differ slightly from anything in the training data. This is the *sparsity problem* in language modelling [JM23], which means that a purely lookup-based approach cannot generalize. It can only repeat what it has already seen.

Before neural networks, the standard way to deal with the sparsity problem was to give up on conditioning on the full sequence and instead condition only on the previous token. This gives the *bigram model*, which was the dominant simple approach to language modelling for many years [JM23]. Rather than computing the full conditional $P(x_i | x_1, \dots, x_{i-1})$, a bigram model approximates it by

$$P(x_i | x_1, \dots, x_{i-1}) \approx P(x_i | x_{i-1}). \quad (12)$$

In words: the probability of the next token depends only on the single token immediately before it. The probability is estimated by counting how often each pair of tokens appears together in the training data and dividing by how often the first token appears at all:

$$P(x_i | x_{i-1}) = \frac{\text{count}(x_{i-1}, x_i)}{\text{count}(x_{i-1})}. \quad (13)$$

A bigram model is also a special case of the more general *n-gram model*, which conditions on the previous $n - 1$ tokens [BDVJ03]. A trigram model uses the previous two, a 4-gram the previous three, and so on. We focus on bigrams in this section because they are the simplest case to reason about and because the limitations they expose carry over to all *n-gram* models.

Bigrams are a real improvement over the full-sequence lookup table from earlier in this section. Instead of $|V|^T$ entries, a bigram model only needs to store probabilities for pairs of tokens. With $|V| = 50,000$, that is at most $|V|^2 = 2.5 \times 10^9$ entries, which is large but actually achievable on real hardware. And many of those pairs never need to be stored explicitly: bigrams that never appear together in training (such as “purple sneeze”) can be treated as having very low probability. In contrast, the bigrams the model does see (*the dog*, *the cat*, *a quick*) cover most of what people actually write.

For a long time, bigram and trigram models were good enough to drive practical systems for tasks such as speech recognition, machine translation, and search. They are far from a toy idea. But they have two limitations that no amount of counting can fix, and these limitations are what eventually motivated neural language models.

The first limitation is that the context window is fixed and very small. A bigram model can only “remember” the single previous token. A trigram model can remember two. Increasing the window further makes the count tables explode in size and the data even sparser, so in practice n rarely exceeds 5. This means a bigram model fundamentally cannot use information from earlier in a paragraph, even when that information is exactly what would tell us what the next word should be. If a sentence begins “The student opened her chemistry textbook and started to read about ...”, the word *chemistry* is essential for predicting what comes next. Still, by the time the bigram model gets to the blank, all it remembers is the word *about*.

The second limitation is more subtle: bigram models have no notion of word similarity. To the model, *cat* and *dog* are completely unrelated symbols, just like *cat* and *xylophone*. If the training data contains many examples of *the cat sat* but not a single example of *the dog sat*, the model has no way to use the first to inform the second, even though any human reader can see they should behave almost identically. Every word lives in its own isolated context, and generalization across similar words does not happen.

Neural language models were developed to address both limitations at once: they replace fixed windows with learned representations that can carry information from arbitrarily far back, and they map words into a continuous space in which similar words are close together, so that what is learned about one word automatically transfers to related ones.

2.1.3 The Neural Network Solution: Manu

If we cannot memorize the language, we have to learn its structure [BDVJ03]. The chain rule already told us that this reduces to modelling the next token given the previous ones,

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1}). \quad (14)$$

For convenience we abbreviate “everything before position t ” as

$$P(x_t \mid x_{<t}), \quad \text{where } x_{<t} = (x_1, x_2, \dots, x_{t-1}). \quad (15)$$

Instead of storing one probability per context, we will compute the probability using a function whose parameters are shared across every context. The cost of the model then grows with the size of the function rather than with the number of possible contexts.

We first need a way to feed text into a function. Since real-valued functions cannot operate directly on words, each token is mapped to a vector of real numbers called a *word embedding* [MCCD13]:

$$x_i \rightarrow \mathbf{e}_i \in \mathbb{R}^d, \quad (16)$$

where d is a small fixed number called the embedding dimension, words that appear in similar contexts tend to mean similar things, so the embeddings are designed (and learned) so that similar words land at nearby points in this d -dimensional space [MCCD13].

The context, now represented as a list of vectors, is fed into a neural network f_θ whose behaviour is controlled by a (large) set of parameters θ . The network outputs one real number for every token in the vocabulary,

$$f_\theta(x_{<t}) \in \mathbb{R}^{|V|}. \quad (17)$$

These numbers, called *logits*, are unnormalized scores indicating how plausible each token is to follow. To turn them into a probability distribution we apply the *softmax* function, which exponentiates each score and normalizes the results to sum to one:

$$P_\theta(x_t \mid x_{<t}) = \text{softmax}(f_\theta(x_{<t})). \quad (18)$$

The model now stores only the parameters θ of a single neural network and computes probabilities on demand. The space requirement drops from $O(|V|^T)$, which Section 2.1.2 showed to be astronomical, to $O(|\theta|)$, which is large but tractable on modern hardware. Because the same parameters are reused across every context, the model can also assign sensible probabilities to sequences it has never seen before. Section 2.2 describes what the function f_θ looks like inside, and how its parameters θ are chosen.

2.2 Basics of a Neural Network: Both

- How it mimics a brain: Manu
- Terminology and main components: Manu/Vanshika
- Linear function, then later advanced to non-linear and why (what the gain is): Manu
- Main algorithms: forward and backward propagation: Vanshika

2.2.1 Neural Networks and the Human Brain: Manu

The function f_θ from the previous section is, in practice, almost always a *neural network* (NN) [GBC16]. Neural networks are a flexible family of parameterized functions, and they get their name from a loose analogy with the brain.

The brain is made up of billions of cells called *neurons*, which receive signals from other neurons, combine them, and pass new signals along. The early researchers who built the first artificial

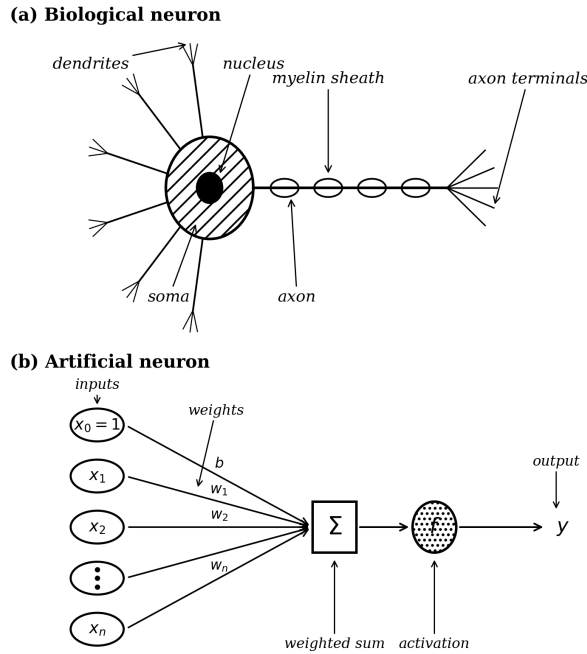


Figure 1: Comparison between a biological neural network (BNN) and an artificial neural network (ANN) [GBC16]

networks asked whether a similarly structured system, composed of simple mathematical units rather than biological cells, could also learn from experience. Figure 1 illustrates the analogy between a biological neural network (BNN) and an artificial neural network (ANN).

A modern neural network has only a few moving parts, and the same components appear in every architecture discussed later in this paper.

- **Neurons.** A neuron is the basic computational unit. It takes in several numbers, computes a weighted sum, adds a constant called a bias, and passes the result through a non-linear function called an activation [GBC16].
- **Connections.** Connections link neurons, and each connection carries a number called a *weight*. The weight controls how strongly one neuron influences the next: a small weight means little influence, a large weight means a lot.
- **Weights and biases.** The weights and biases are the *parameters* of the network, the numbers that get adjusted during training. Weights set the strength of each connection; biases shift a neuron’s output up or down. To see why a bias is useful, consider a function such as $y = \sin(x) + b$: changing b moves the curve up or down without changing its shape, which gives the model an extra degree of freedom for fitting data.
- **Activation functions.** Activation functions are the small nonlinear functions applied at the end of each neuron. They are what allow a network to represent more than straight-line relationships. Common choices are the sigmoid, the hyperbolic tangent, and the rectified linear unit (ReLU). Section 2.2.2 explains why this non-linearity is essential.
- **Learning rule.** A learning rule specifies how the weights and biases are updated when the network makes mistakes. In modern neural networks, this is a gradient-based optimization method, such as stochastic gradient descent [GBC16]; we describe this in detail in Section 2.2.4.

The brain analogy should not be taken too literally. Biological neurons receive electrical and chemical signals from other neurons through structures called *dendrites*, combine those signals inside the cell body, and emit a spike along an outgoing fibre called an *axon* when the combined

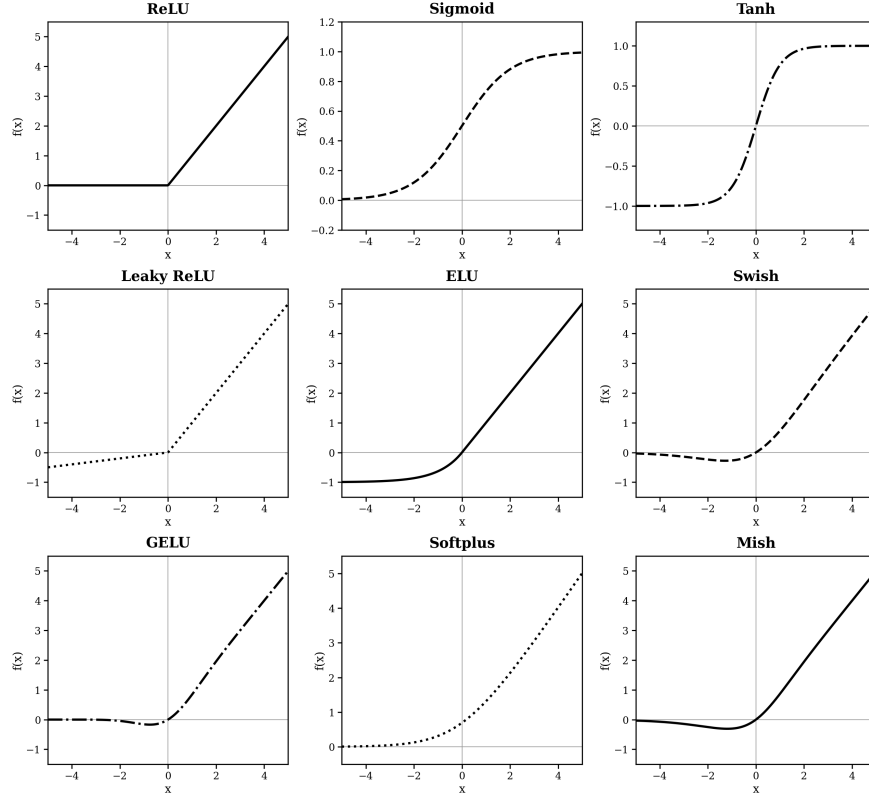


Figure 2: Examples of commonly used activation functions [GBC16]

signal crosses a threshold. Artificial neurons follow the same general recipe of combining inputs and producing an output, but the underlying mechanics are very different. The mechanisms of learning in the human brain are still not fully understood, while artificial neural networks are trained using well-defined mathematical optimization. The biological story is best read as historical motivation rather than a literal claim about either system [GBC16].

2.2.2 Why Non-Linear Activation Functions Are Necessary: Manu

Section 2.2.1 introduced activation functions and noted that they have to be non-linear, but it did not say why. The reason is that without non-linear activations, stacking more layers does not make a neural network any more powerful. A deep stack of linear layers is mathematically equivalent to a single linear layer.

The first artificial neural network model, the **Perceptron**, was introduced by Frank Rosenblatt in 1957 [GBC16]. A perceptron computes a weighted sum of its inputs and applies a step function: if the sum is above zero, the output is 1, otherwise it is 0:

$$y = \begin{cases} 1 & \text{if } w^\top x + b > 0 \\ 0 & \text{otherwise.} \end{cases} \quad (19)$$

A perceptron can solve any problem whose answer can be drawn as a single straight line separating two classes, but many real problems are not like this. The standard counter-example is the XOR problem, where no single straight line can correctly separate the inputs (0,0) and (1,1) from (0,1) and (1,0) [GBC16]. To handle problems like XOR, researchers introduced **Multi-Layer Perceptrons (MLPs)**, which add one or more hidden layers between the input and the output and apply a non-linear activation at each layer. The *Universal Approximation Theorem* provides the theoretical justification for this: a feedforward neural network with at least one hidden layer and a non-linear activation function can approximate any continuous function on a compact subset of $\mathbb{R}^n$, given enough neurons [GBC16].

Why depth without non-linearity adds nothing.

The easiest way to see why non-linearity is necessary is to do the algebra. Suppose we stack two purely linear layers, with no activation function in between:

$$h = W_1x + b_1, \quad (20)$$

$$y = W_2h + b_2. \quad (21)$$

Substituting the first equation into the second gives

$$y = W_2(W_1x + b_1) + b_2, \quad (22)$$

which expands to

$$y = W_2W_1x + W_2b_1 + b_2. \quad (23)$$

Defining a new weight matrix and bias,

$$W' = W_2W_1, \quad (24)$$

$$b' = W_2b_1 + b_2, \quad (25)$$

The two-layer network reduces to

$$y = W'x + b'. \quad (26)$$

This is just a single linear layer. The same trick works for any number of stacked linear layers: they can always be rewritten as one linear layer with a different weight matrix and bias. A single linear layer can only represent straight-line relationships, so depth on its own provides no additional representational power.

The effect of adding a non-linearity.

Inserting a non-linear activation $\sigma(\cdot)$ between the two layers gives

$$h = \sigma(W_1x + b_1), \quad (27)$$

$$y = W_2h + b_2. \quad (28)$$

Because σ is non-linear, there is no algebraic way to merge W_1 and W_2 into a single matrix, and the two layers do separate work. The network can now represent curved, multi-region decision boundaries that no linear model can capture, which is why every modern neural network includes activation functions between its linear layers.

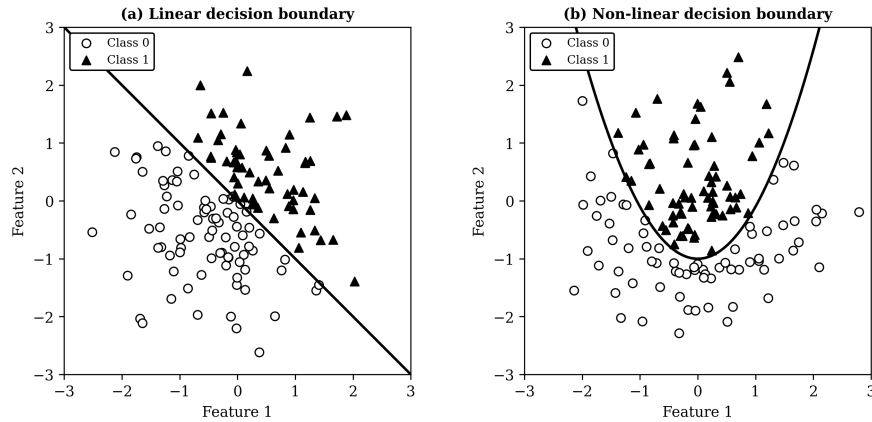


Figure 3: Comparison of linear and non-linear decision boundaries [GBC16]

Feedforward Neural Networks.

Multi-Layer Perceptrons are the simplest example of a more general family called **Feedforward Neural Networks (FFNNs)**. The defining property of an FFNN is that information flows in a single direction, from the input layer through the hidden layers and out the output layer, with no loops or feedback connections.

A forward pass through an FFNN proceeds as follows:

1. Input data is fed into the input layer.
2. Each layer applies a linear transformation followed by an activation function.
3. The result is passed through one or more hidden layers in turn.
4. The final layer produces a prediction.

The structure of the final layer depends on the task:

- **Regression.** For predicting continuous numerical values such as prices or temperatures, the output layer is a single neuron with no activation function, allowing it to produce any real number.
- **Binary classification.** For yes/no problems, the output layer is a single neuron with a sigmoid activation, which maps its input to a probability between 0 and 1.
- **Multi-class classification.** For problems with more than two classes (such as language modelling, where the classes are the vocabulary tokens), the output layer has one neuron per class, and a softmax activation converts the outputs into a probability distribution that sums to 1.

The multi-class case with a softmax output is exactly the setup needed to compute $P_\theta(x_t | x_{<t})$ for next-token prediction, as defined in Section 2.1.3.

2.2.3 FFNN intro and architecture - Vanshika

As we already discussed above how MLPs are the same as feed-forward neural networks (FFNNs), where data flows forward without loops, and they can have one or more hidden layers [GBC16]. Learning in FFNN is basically a process of optimizing the parameter set $\theta = \text{Weights and Biases}$ and to minimize a cost function $J(\theta)$ [GBC16].

2.2.4 Main Algorithms: Forward and Backward Propagation- Vanshika

In FFNN, learning can be done using two algorithmic passes: the forward pass, which is used to generate a prediction, and the backward pass, which is used to compute the necessary gradients for optimization.[GBC16].

Forward Propagation In the forward pass, input data is passed from the input layer, through one or more hidden layers, to the output layer, Each layer takes the output of the previous one, transforms it, and passes it forward until a final cost is calculated. For a given layer l , the linear transformation and then non-linear activation are computed as shown below:

$$z^{[l]} = W^{[l]}x^{[l-1]} + b^{[l]} \quad (29)$$

$$x^{[l]} = \sigma(z^{[l]}) \quad (30)$$

where $W^{[l]}$ is the weight matrix, $b^{[l]}$ is the bias vector, and σ is a non-linear activation function (such as ReLU or sigmoid). This process continues until the final output $\hat{y}$ (predicted outcome) is generated, which is then used to compute the total scalar cost J :

$$J = L(\hat{\mathbf{y}}, \mathbf{y}) + \lambda \Omega(\theta) \quad (31)$$

In this expression, L is the loss function, which could be a MSE measuring the error between the prediction $\hat{\mathbf{y}}$ and the true target $\mathbf{y}$ from our dataset, while $\Omega(\theta)$ is a regularization term scaled by the hyperparameter λ to prevent overfitting of the parameters θ . [GBC16].

Backward Propagation. After the forward pass, to minimize the loss, the network adjusts its parameters W and b to reduce J [GBC16].

Output Error We begin by finding how the cost changes with respect to the output:

$$\mathbf{g} = \nabla_{\hat{\mathbf{y}}} J \quad (32)$$

This gradient $\mathbf{g}$, is the *error signal* flowing backward from the cost to the inputs. [GBC16].

Back-propagating through Layers For each layer l , from L to 1, this error signal passes through the non-linearity and linear transformation.

- Gradient on Pre-activation

$$\mathbf{g} \leftarrow \mathbf{g} \odot \sigma'(\mathbf{z}^{(l)}) \quad (33)$$

- Gradient on Parameters

$$\nabla_{\mathbf{b}^{(l)}} J = \mathbf{g} \quad (34)$$

$$\nabla_{\mathbf{W}^{(l)}} J = \mathbf{g}(\mathbf{a}^{(l-1)})^\top \quad (35)$$

- Gradient on Previous Layer

$$\mathbf{g} \leftarrow (\mathbf{W}^{(l)})^\top \mathbf{g} \quad (36)$$

The symbol $\odot$ denotes the Hadamard product (element-wise multiplication).

These gradients determine how the weights and biases are then updated using an optimization algorithm, such as Stochastic Gradient Descent (SGD),

$$W^{[l]} \leftarrow W^{[l]} - \alpha \frac{\partial L}{\partial W^{[l]}} \quad (37)$$

where α represents the learning rate, a hyperparameter that controls the step size of the update [GBC16].

2.3 How is the Decoder-Only Transformer Architecture different from FFNN : Vanshika

For our transformer, we will be using decoder only transformers. encoder decoder ... We saw that FFNN in Section 2.2 use fixed layer connections; however, Decoder-Only Transformer evolves this concept by using Self-Attention mechanism[VSP+17], which allows the model to assign different levels of importance to different tokens within an input sequence when generating each output. This enables context awareness understanding in the input Embeddings . For instance, in “The cat sat on the mat because it was soft,” the self-attention enables the model to determine that “it” is referring to “the mat.”

The Transformer uses masked self-attention to capture interactions between all pairs of tokens in a sequence. This creates direct pathways between every token and every earlier token while still preventing access to future positions, and it allows highly parallel computation during training[VSP+17]. while also maintaining autoregressive flow, where it begins with a start token and uses the list of previous outputs as its inputs, which contain all the attention information from the first input. [VSP+17].

A single decoder block contains the following key components like in the figure:

Masked Multi-Head Self-Attention:

Unlike standard layers in section 2.2, the attention mechanism here allows the future tokens to attend to the prior tokens to gain contextual understanding. Adding Mask M, allows the model to not see or get influenced by future tokens during training [VSP+17]. Formally, attention is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V \quad (38)$$

where Q, K, V are the Query, Key, and Value matrices, d_k is the dimensionality of the key vectors, and M is a mask matrix that assigns $-\infty$ to positions corresponding to future tokens, ensuring that the model cannot attend to them during training.

2.4 Pre-Training versus Post-Training: Manu

2.4.1 Pre-training vs Post-training

The previous sections explained what a language model represents and how a neural network can be used to compute it. They did not explain where the parameters of that network actually come from. In practice, building a modern Large Language Model (LLM) is split into two phases with very different goals: *pre-training* and *post-training*.

Pre-training gives the model its general capabilities: its grasp of grammar, its broad world knowledge, its sense of which words tend to follow which others, and its ability to perform basic reasoning. During pre-training, the model is exposed to large amounts of unlabelled text and learns the statistical patterns of language by repeatedly predicting the next token, exactly as described in Section 2.1.3. No human is labelling each example; the text itself supplies the answer (the next token), which is why pre-training is described as *self-supervised*.

Post-training refines how the model uses what it has learned. After pre-training, a model can produce fluent, coherent text, but it does not necessarily produce *helpful* text, and it does not necessarily refuse to produce harmful text. Post-training addresses this by guiding the model to follow instructions, maintain an appropriate tone, stay on task, and respect safety constraints. It typically uses much smaller, carefully curated datasets and human feedback.

The relationship between the two stages can be thought of as nature and nurture: pre-training gives the model its foundational capabilities, and post-training shapes how those capabilities are expressed in practical interactions.

Key differences.

- **Pre-training:** learning general language patterns and world knowledge from large unlabelled datasets through next-token prediction.
- **Post-training:** adjusting the model’s behaviour, instruction-following, tone, and safety using smaller curated datasets and human feedback.

This thesis focuses entirely on pre-training, since our goal is to study how the choice of pre-training corpus shapes the foundational knowledge and biases of the resulting model. We do not perform any post-training. Before moving on to implementation details in Section 3, this subsection walks through what a pre-training pipeline looks like at a high level.

High-level steps in pre-training.

1. **Dataset collection.** A large body of text is gathered from diverse sources such as books, websites, and articles. Since the model can only learn from what it sees, the choice of sources is one of the most consequential decisions in the whole pipeline.
2. **Data processing and tokenization.** The raw text is cleaned, filtered, and converted into tokens by a tokenizer that maps words or subwords to integer IDs, the same tokens defined in Section 2.1.1.
3. **Model initialization.** The neural network architecture (in our case, a decoder-only transformer) is initialized with small random weights.
4. **Self-supervised training.** The model is repeatedly shown chunks of the tokenized text and asked to predict the next token at every position. Because the correct answer is always the next token in the actual text, no manual labelling is required.
5. **Parameter optimization.** A gradient-based optimizer (such as the stochastic gradient descent method introduced in Section 2.2.4) updates the model’s parameters to reduce the prediction error, one batch at a time, until the corpus is exhausted.

Section 3 revisits each of these stages and describes how we carry them out for our two philosophical corpora, including the architectural choices, training infrastructure, and the procedure for comparing Eastern and Western training data.

3 Method

3.1 Pre-Training Steps: Vanshika

3.1.1 Architecture and Tokenization

We selected **GPT-2** as our baseline model because it is open source and follows the standard decoder-only transformer design with traditional Multi-Head Attention and core autoregressive properties [RWC⁺19]. This architecture is also well understood and widely studied, which lets us evaluate our results when training from scratch on philosophical text. We chose GPT-2 with three distinctions that improve deep-layer convergence:

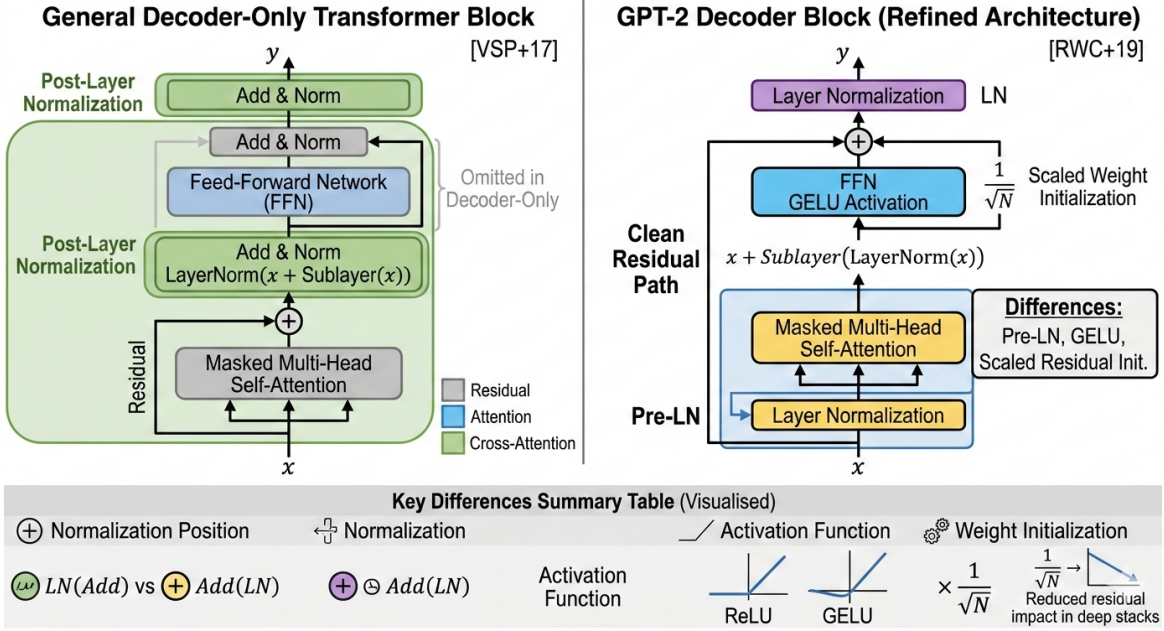


Figure 5: Comparison between a vanilla decoder and the modified version used by GPT-2, from Vaswani et al. [VSP⁺17] and [RWC⁺19]

- **Pre-Layer Normalization:** Unlike the standard decoder, which applies normalization after the residual addition (Post-LN) [VSP⁺17], GPT-2 moves Layer Normalization to the input of each sub-block (Pre-LN) [RWC⁺19]. This creates a cleaner identity path for gradients, which is essential when training the model’s deeper variants.
- **Activation Function:** As shown in Figure 3, GPT-2 uses Gaussian Error Linear Units (GELU) [RWC⁺19] instead of the ReLU activation used by vanilla decoder-only transformers [VSP⁺17]. GELU provides a smoother, non-monotonic curve that has been shown to improve performance in high-complexity language tasks [HG16].

$$\text{Original FFN: } \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

$$\text{GPT-2 FFN: } \text{GELU}(xW_1 + b_1)W_2 + b_2$$

- **Modified Residual Initialization:** The weights of the residual layers are scaled initially by a factor of $1/\sqrt{N}$, where N is the total number of layers [RWC⁺19]. This accounts for the natural buildup of variance in the residual connections as they move through the stack. By making each residual layer contribute a little less at the start, the model keeps the variance stable across the network, which makes the gradient move more smoothly during backpropagation [RWC⁺19, XYH⁺20].

GPT-2 Scaffold: we use the `tiktoken` library for efficient Byte-Pair Encoding (BPE). We initialize with random weights to ensure that the results are derived exclusively from our training set.

3.1.2 Data Formatting and Pipeline

To ensure consistent ingestion, the training corpus is pre-processed to UTF-8 plain text and stored in a centralized `data/` directory.

- **Sharding Strategy:** the corpus is split into sequential shards (e.g. `shard-0001.txt`, `shard-0002.txt`).
- **Concatenation:** during initialization, the training script (`gpt2_pretrain.py`) ingests these files in sorted alphanumeric order. The text is concatenated into a single continuous stream, allowing the model to learn dependencies that span individual file boundaries.

3.1.3 Initialization and Training Loop

Unlike fine-tuning approaches, the model is initialized with random weights (a “from scratch” model) so that all learned linguistic and philosophical representations are derived solely from our Western and Eastern corpora.

Training was executed on the shared GPU server described in Section 3.5.2, using two named `tmux` sessions to run the Eastern and Western jobs concurrently without blocking each other. Each session was detached immediately after launch so that jobs continued running across SSH disconnections. The Eastern model was pinned to GPU 0 and the Western model to GPU 1 via the `CUDA_VISIBLE_DEVICES` environment variable, ensuring no memory contention between runs.

3.2 Data Collection: Manu

To evaluate whether models trained on distinct philosophical traditions develop measurably different linguistic and conceptual biases, we curated two large-scale corpora: a **Western** corpus spanning Greco-Roman, Christian, and modern European philosophy, and an **Eastern** corpus covering Chinese, Indian, Japanese, Korean, and Tibetan philosophical traditions. In total the source catalogue contains 378 entries, of which approximately 181 were successfully downloaded as public-domain English text. The remaining 197 are documented with reasons for unavailability (e.g. no public-domain English translation, copyright restrictions, or lost primary works).

3.2.1 Sourcing Strategy: Manu

All texts were collected using an automated download pipeline (`download_pretraining_data.py`) that draws from five public-domain sources:

- **Project Gutenberg:** used for high-quality, boilerplate-stripped plain-text editions of classical works (e.g. Griffith’s *Rigveda*, Jowett’s Plato, Legge’s Confucius). Both direct downloads by book ID and keyword-based search via the Gutendex API were employed.
- **Internet Archive:** used for academic and historical translations not available on Gutenberg, retrieved via the Advanced Search JSON API with automatic selection of the best available `.txt` file per item.
- **SuttaCentral:** used for Bhikkhu Sujato’s modern English translations of the Pali Canon (Dīgha, Majjhima, Saṃyutta, Aṅguttara, Khuddaka Nikāyas, and Dhammapada), accessed through the bilara-data API.
- **Access to Insight:** used for additional Theravāda Buddhist suttas, collected via recursive HTML scraping of the Tipiṭaka index pages.
- **Sacred-Texts.com:** used selectively for texts not available through the above sources, collected via subpage traversal and HTML-to-text conversion.

Every downloaded text passes through a post-processing pipeline that (i) strips Project Gutenberg boilerplate headers and footers, (ii) converts HTML to plain text where necessary, (iii) normalizes whitespace and line breaks, and (iv) applies an English-language filter based on ASCII ratio, Latin-alphabet ratio, and stopword frequency to reject non-English or heavily corrupted downloads.

3.2.2 Temporal Alignment and Corpus Composition: Manu

A critical challenge in comparative philosophy is separating cultural differences from temporal ones. Differences often attributed to “East vs. West” may actually be driven by historical era or translation style. To address this, both corpora are organized into 21 chronological time periods: one pre-Common Era folder (`older` (BC)) and twenty 100-year intervals from 100 CE through 2000 CE. Each text is assigned to a period based on the earliest known composition date of the original work, not the date of translation. This structure enables the progressive training procedure described in Section 3.6 and lets us control for temporal confounds when comparing Eastern and Western outputs.

The Western Corpus spans approximately 142 philosophers and covers the full arc of European and Mediterranean thought. Representative texts include:

- **Classical Antiquity:** Pre-Socratic fragments, Plato’s dialogues (Jowett translations), Aristotle’s major works, Epicurus, Lucretius’s *De Rerum Natura*, Cicero, Seneca, Epictetus’s *Discourses*, and Marcus Aurelius’s *Meditations*.
- **Late Antiquity and Medieval:** Plotinus’s *Enneads*, Augustine’s *Confessions* and *City of God*, Boethius’s *Consolation of Philosophy*, Anselm’s *Proslogion*, Aquinas’s *Summa Theologiae*, as well as Islamic-tradition philosophers classified under the Western canon (Al-Kindi, Avicenna, Al-Ghazali).
- **Early Modern to Modern:** Descartes, Spinoza, Locke, Hume, Kant, Hegel, Kierkegaard, Nietzsche, William James, Bertrand Russell, Wittgenstein, Heidegger, Sartre, Camus, and Simone de Beauvoir, among others.

Also included is the King James Version of the Bible (1611) as a foundational Western religious text.

The Eastern Corpus spans approximately 215 entries across five geographic traditions:

- **Chinese Philosophy** (≈ 96 entries): Confucian classics (Analects, Mencius, Xunzi), Daoist texts (Tao Te Ching, Zhuangzi, Liezi), Legalist works (Han Feizi, Shang Yang), Mohist texts, Neo-Confucian writings (Zhu Xi, Wang Yangming), and Chan/Zen texts (Platform Sutra, Huineng).
- **Indian Philosophy** (≈ 78 entries): Vedic texts (Rigveda, Upanishads), the Pali Canon (six collections via SuttaCentral), the Bhagavad Gita, Jain sutras (Jacobi’s SBE translations), Nyāya, Sāṅkhya, and Yoga sūtras, as well as Mahāyāna Buddhist works (Nāgārjuna, Vasubandhu, Asaṅga).
- **Japanese Philosophy** (≈ 16 entries): Zen Buddhist texts (Dōgen, Hakuin), Shinto classics (Kojiki, Nihon Shoki), and Bushidō literature.
- **Korean Philosophy** (≈ 10 entries): Neo-Confucian works (Yi Hwang, Yi I) and Korean Buddhist texts (Wonhyo, Jinul).
- **Tibetan Philosophy** (≈ 14 entries): texts from Milarepa, Tsongkhapa, Gampopa, and Longchenpa.

3.2.3 Handling Unavailable Texts: Manu

Of the 378 catalogued entries, 197 could not be downloaded because no public-domain English translation exists, the only available translations remain under copyright, the primary work survives only as fragments within other texts already in the corpus, or access was restricted by the hosting platform. Each unavailable entry is documented in the source catalogue with a machine-readable reason code (e.g. `no_pd_english`, `copyright_trans`, `lost_work`, `restricted_ia`), ensuring full reproducibility and transparency about corpus coverage gaps.

3.3 Environment Setup: Manu

All training and evaluation runs were carried out on **cmeps03**, a shared GPU server hosted by the UBC Okanagan Department of Computer Science. The machine is equipped with two **NVIDIA RTX 6000 Ada Generation** GPUs (48 GB VRAM each), running Ubuntu Linux with CUDA 12.x. Training jobs were launched on a single GPU at a time to keep memory headroom for the 124M-parameter GPT-2

architecture at a 1024-token context length, with the second GPU reserved for concurrent evaluation runs.

The software stack consists of Python 3.10 with PyTorch, the Hugging Face `transformers` and `datasets` libraries, the `tiktoken` tokenizer, and a small in-house `lm_utils` module that wraps dataset construction, blockification, and the `Trainer` loop. All artifacts (model checkpoints, tokenizer configurations, training logs, and evaluation JSONs) are written under `~/outputs_full/` on the server, organized by region and time period (e.g. `progressive_west/period.2000/checkpoint-2118`).

The full progressive training pipeline (21 sequential time periods, separately for the Western and Eastern corpora) takes on the order of 24–36 hours of wall-clock time per region on a single RTX 6000 Ada, with most of that time spent on the later, larger time periods. Evaluation runs across all 14 prompt categories and 284 prompts, completing in roughly 30–45 minutes per checkpoint.

3.4 Evaluation Metrics: Vanshika

To evaluate whether the two models had developed distinct philosophical biases, we constructed a fixed set of 284 prompts organised into 14 thematic categories. The categories were chosen to span the major areas of philosophical inquiry that both Eastern and Western traditions address, while also including several areas where one tradition has developed highly specific technical vocabulary with no direct equivalent in the other. Table 1 lists the 14 categories and the number of prompts in each.

Table 1: Evaluation prompt categories and prompt counts.

Category	Prompts
Self Identity	20
Purpose Meaning	20
Ethics Morality	20
Reality Existence	20
Knowledge Truth	20
Death Immortality	20
Nature Universe	20
Enlightenment Liberation	20
Free Will Fate	20
Good Evil	20
Society Justice	20
Death Meaning	20
Body Mind	20
Wisdom Truth	24
Total	284

Each prompt is a short open-ended philosophical question such as “*What is the self?*”, “*What gives life meaning?*”, “*How does one achieve liberation?*”, or “*What is the relationship between mind and body?*” Prompts were written to be tradition-neutral in phrasing—that is, they do not use vocabulary specific to either Eastern or Western philosophy, so that any tradition-specific vocabulary in the model’s response must originate from the training corpus rather than from the prompt itself.

The 14-category structure serves two purposes. First, it provides robustness: any single prompt might produce an anomalous result due to decoding randomness, but consistent patterns across 20 prompts within a category constitute a reliable signal. Second, the categories vary in how sharply the two traditions diverge in vocabulary. Categories such as *Enlightenment Liberation* and *Body Mind* are expected to show large Eastern marker ratios in the Eastern model’s outputs because Eastern traditions developed specific technical vocabulary for these concepts (nirvana, moksha, shunyata, dukkha, anatta) with no Western equivalent. Categories such as *Free Will Fate* and *Society Justice* are expected to show smaller gaps because both traditions discuss these topics using overlapping vocabulary. This variation across categories lets us test not just whether bias exists overall, but whether its strength correlates with the degree of vocabulary specialisation in each tradition.

For each prompt, both the Western and Eastern models generated a response using the fixed decoding configuration described in Section 3.4.1. All 284 prompts were held constant across every checkpoint evaluated, ensuring that differences in results across runs reflect model behaviour

rather than prompt variation. Evaluation outputs were serialised to timestamped JSON files under `progressive evaluations/` so that every run is fully reproducible.

3.4.1 Decoding Configuration and Iterative Tuning- vanshika

The fixed decoding configuration used for all evaluations in this thesis — `temperature=0.4`, `top_p=0.9`, `repetition_penalty=1.3`, `no_repeat_ngram_size=4`, was arrived at through iterative experimentation across multiple checkpoint steps and parameter values. The process is described here because it directly affected which checkpoints were usable and why the fixed-step and progressive runs were evaluated the way they were.

Baseline behaviour without constraints. The earliest evaluations used greedy decoding with no repetition controls. At all checkpoint steps below 5,000, both models entered degenerate repetition loops immediately — repeating the same phrase ten or more consecutive times before the token limit was reached. For example, the Eastern model at 2,000 steps with greedy decoding produced output such as:

“How could a person be born? How could a person be born? How could a person be born? ...”

This behaviour made all evaluation metrics meaningless: a looping model produces artificially stable perplexity on its own outputs and inflated repetition scores that swamp any bias signal.

Temperature. We tested `temperature` $\in \{1.0, 0.8, 0.7, 0.5, 0.4\}$. At 1.0, outputs were incoherent — the model sampled low-probability tokens too freely. At values below 0.4, the distribution collapsed back toward the highest-probability token at each step, causing repetition to re-emerge. `temperature=0.4` was the lowest value that preserved philosophical coherence without reintroducing loops.

Top-p (nucleus) sampling. We tested `top_p` $\in \{1.0, 0.95, 0.9, 0.85\}$. At 1.0, occasional very low-probability tokens caused incoherent topic shifts. `top_p=0.9` sampled from the top 90% of the probability mass, which included all plausible continuations while excluding noise tokens, and was retained for all final evaluations.

Repetition penalty. The repetition penalty multiplicatively reduces the log-probability of any token that has already appeared in the current output. We tested penalty values $\in \{1.0, 1.1, 1.2, 1.3, 1.5\}$. At 1.0 (no penalty), looping persisted at all checkpoints. At 1.1 and 1.2, looping was reduced but not eliminated. At 1.3, looping was fully eliminated across all 284 prompts and both models. At 1.5, the penalty was too aggressive: the model was forced away from contextually appropriate vocabulary, producing incoherent topic shifts mid-response. `repetition_penalty=1.3` was selected.

N-gram repetition block. `no_repeat_ngram_size=4` prevents any 4-token sequence from appearing more than once in a generated output. This hard constraint acts as a backstop for any repetitive patterns the soft penalty misses. N-gram sizes of 3 and 5 were also tested. Size 3 was too aggressive on philosophical texts that naturally repeat short phrases such as *“the nature of”* or *“in relation to”*. Size 5 allowed some repetition to slip through. 4-gram blocking provided the right level of constraint without distorting natural language patterns.

Checkpoint selection In addition to tuning decoding parameters, we evaluated intermediate checkpoints at 500, 1,000, 2,000, 5,000, and 15,000 steps for the fixed-step models. Checkpoints at 500 and 1,000 steps produced incoherent output even with full decoding constraints, because the model had not yet converged on a stable vocabulary distribution. The 5,000-step checkpoint was the earliest point at which outputs were philosophically coherent and showed a measurable bias signal. The 15,000-step checkpoint produced the best surface-level fluency but the worst cross-perplexity (Section 4.1.2), confirming that surface quality and distributional divergence are not the same thing and that more training does not monotonically improve evaluation results.

The final decoding configuration was fixed before any bias evaluation results were examined, ensuring that it was not chosen to favour any particular model or checkpoint. All results reported in Section 4.2.3 use this identical configuration.

3.4.2 Cross-Perplexity (Primary Metric): Manu

Our primary metric is **cross-perplexity**: the perplexity assigned by one model to text generated by the other. Specifically, we compute PPL_{west} (east-output) and PPL_{east} (west-output). If the two models have learned distinct linguistic and conceptual distributions, each should find the other’s output highly improbable, yielding large cross-perplexity values. If they have converged on similar distributions, cross-perplexity will be low. This formulation lets us quantify model divergence directly from output behaviour without relying on a shared held-out test set.

3.4.3 Lexical and Stylistic Metrics: Manu, Vanshika

To complement cross-perplexity, we report:

- **Type-Token Ratio (TTR)**: the ratio of unique tokens to total tokens in generated outputs, used as a coarse measure of lexical diversity. Low TTR values indicate that a model is collapsing into a small repeated vocabulary, a known failure mode of overfitting small language models.
- **Repetition rate**: the fraction of repeated n -grams in generated text, used together with TTR to detect degenerate outputs that would otherwise inflate perceived novelty.

These two metrics were essential during model selection. The fixed-step runs at 5,000 and 15,000 steps (Section 3.5.1) provided the empirical baseline: the 5,000-step checkpoints showed acceptable repetition scores (0.000–0.003) and type-token ratios (0.80–0.86), but generated degenerate repetitive output at inference time without decoding constraints; the 15,000-step checkpoints showed only marginal improvements in TTR (0.84–0.89), while exhibiting extreme cross-perplexity inflation (W→E: 100,000–285,000), indicating distribution memorisation rather than meaningful generalization. The progressive pipeline’s epoch cap was introduced specifically to avoid both failure modes, and the addition of a repetition penalty (`repetition_penalty=1.3`, `no_repeat_ngram_size=4`) at inference time resolved generation looping without requiring any additional training.

3.4.4 Concept Marker Frequency: Manu

To test whether each model has internalized the conceptual vocabulary of its training tradition, we maintain two curated marker lists:

- A **Western marker list** containing terms such as *soul*, *virtue*, *reason*, *logos*, *justice*, *substance*, *categorical*, *dialectic*, *Kant*, *Hegel*, *Aristotle*, etc.
- An **Eastern marker list** containing terms such as *dharma*, *karma*, *nirvana*, *atman*, *brahman*, *tao*, *qi*, *wu wei*, *sunyata*, *dukkha*, *anatta*, *mindfulness*, etc.

For each generated response, we compute the relative frequency of Western vs. Eastern markers and aggregate per category. The expected pattern is that the Western model produces a significantly higher Western-marker ratio than the Eastern model, and vice versa. Asymmetries in this ratio are reported per prompt category to surface where each tradition dominates the generated output.

3.4.5 Distributional Overlap: Manu

Finally, to characterize the global similarity of the two models’ output distributions, we compute the **Bhattacharyya coefficient** and **Bhattacharyya distance** between the empirical word distributions of Western and Eastern generations, along with a **KL-divergence** report (`kl_divergence.py`) over high-frequency vocabulary. These distributional measures provide a single corpus-level summary that complements the per-category breakdown above and is interpreted on a coarse five-level scale from “very high overlap” to “very low overlap.”

All evaluation outputs are serialized to timestamped JSON files (e.g. `bias_evaluation_20260410_062642.json`) under `progressive_evaluations/` for downstream analysis and figure generation.

3.5 Fixed-Step Training- vanshika

3.5.1 Motivation and Contrast with Progressive Training

The fixed-step approach trains a single model per region on the full concatenated corpus, without temporal decomposition. Where the progressive pipeline calls `load_cumulative_texts` to accumulate data period by period and resets the model for each period, the fixed-step script simply concatenates every `.txt` file in `data/east/` or `data/west/` into one continuous token stream and trains until a hard step limit is reached.

This simplicity is both the appeal and the limitation of the fixed-step approach. It is easy to run and easy to reproduce, but the step count is not calibrated to corpus size. A fixed 5,000-step budget over-trains a small corpus and under-trains a large one; the progressive pipeline was designed specifically to avoid this.

3.5.2 Training Configuration

Both Western and Eastern fixed-step models are initialised using `build_gpt2_from_scratch` (in `gpt2_pretrain.py`), which constructs a GPT-2 config and model with random weights:

- **Architecture:** 12 layers, 12 attention heads, 768 embedding dimensions, 1,024-token block size — identical to the progressive pipeline models.
- **Tokenizer:** tiktoken gpt2 encoding; vocabulary size = `encoding.n_vocab`; EOS token (`encoding.eot_token`) used as sequence separator and pad token.
- **Dataset mode:** streaming (`StreamingLMDataset`) for large corpora to avoid loading all text into memory; non-overlapping blocks of 1,024 tokens with an approximate shuffle buffer.
- **Optimiser:** AdamW (fused), learning rate 5×10^{-4} , weight decay 0.1, 100 warmup steps.
- **Batch size:** per-device batch size 8, gradient accumulation 4 steps, effective batch size 32.
- **Precision:** bf16 where supported by the GPU (RTX 6000 Ada supports bf16); fp16 fallback otherwise.

The key difference from the progressive pipeline is the step budget. Rather than `compute_max_steps` scaling steps to corpus size, the fixed-step runs use a single command-line argument (`--max-steps`) set manually:

```
# Eastern model | GPU 0
CUDA_VISIBLE_DEVICES=0 python3 gpt2_pretrain.py \
  --subdir east \
  --output-dir ./outputs/eastern_model_v2 \
  --streaming \
  --max-steps 5000      # or 15000 for extended run

# Western model | GPU 1
CUDA_VISIBLE_DEVICES=1 python3 gpt2_pretrain.py \
  --subdir west \
  --output-dir ./outputs/western_model_v2 \
  --streaming \
  --max-steps 5000
```

Both jobs were managed using named `tmux` sessions (`tmux new -s east` and `tmux new -s west`) so they ran concurrently on separate GPUs and continued across SSH disconnections. Checkpoints were written every $\sim 10\%$ of the step budget to `outputs/eastern_model_v2/` and `outputs/western_model_v2/` respectively.

3.5.3 What the Fixed-Step Pipeline Does Not Do

For completeness, the fixed-step pipeline omits several features that the progressive pipeline includes:

- No dynamic step budget — the same step count applies regardless of corpus size.
- No epoch cap — the model can make arbitrarily many passes over a small corpus if `max-steps` is large enough relative to the data volume.
- No resume support — if a run is interrupted, it must be restarted from scratch (or from the most recent checkpoint using HuggingFace Trainer’s built-in resume).

These omissions are acceptable for the fixed-step experiments because each run is a single job rather than a 21-period pipeline. They become problems at scale, which is why the progressive pipeline adds all of them.

3.6 Progressive Training Pipeline: Manu

The actual training is driven by `main_progressive_training.py`, which trains the Western and Eastern GPT-2 models in lockstep across the 21 chronological time periods defined in Section 3.2. This subsection describes how that pipeline works and why each piece is there.

3.6.1 Why Progressive, Not Single-Shot

While the fixed-step baselines described in Section 3.5 train a single model on the full corpus, the progressive pipeline adopts a different approach: it trains 21 separate models per region, where the model for period p is trained on all text from period p and earlier. The model for *older* (BC) sees only pre-Common Era texts, the model for period *100* sees BC plus 0–100 CE texts, and so on up through period *2000*, which sees everything.

There are two reasons to do it this way. Each period yields its own checkpoint and evaluation, so we get a series of snapshots of how the bias signal develops over time, not just a single endpoint. And small early corpora and large late corpora go through the same code path, which forces the training-budget logic to scale with corpus size instead of being hard-coded for one particular size.

3.6.2 Per-Period Training Loop

For each time period, and for each region (east and west), the pipeline does the following:

1. **Load cumulative texts.** `load_cumulative_texts` walks every period directory from *older* (BC) up to the current period and concatenates all the texts under `data/{region}/{period}/`. The data layout from Section 3.2 is what makes this a one-line operation.
2. **Build a fresh model from scratch.** A new GPT-2 (12 layers, 12 heads, 768 embedding dimensions, 1024 block size) is constructed using `build_gpt2_from_scratch`, with random weights and a fresh tiktoken-based tokenizer. We do not warm-start from the previous period’s checkpoint, because doing so would conflate “the model learned this from earlier texts” with “the optimizer state was already partway to a solution.”
3. **Compute the step budget.** The number of training steps is set dynamically by `compute_max_steps`, which scales linearly with corpus size at 20 steps per text and clamps the result to the range [300, 5000]. The early periods, which have only a handful of texts, get a few hundred steps; the late periods, which have hundreds of texts, hit the 5,000-step cap.
4. **Apply the epoch cap.** After the dataset has been built, `compute_epoch_capped_steps` computes how many steps would constitute three full passes over the dataset and reduces `max_steps` to that ceiling if necessary. This is the cap discussed in Section 5: it was added after early small-corpus runs at 5,000 and 15,000 steps memorized their training data verbatim instead of learning a distribution.

5. **Train via Hugging Face Trainer.** `build_trainer` (in `lm_utils.py`) wraps the model, dataset, and collator in a standard `Trainer` configured with the hyperparameters from `DEFAULT_CONFIG` (learning rate 5×10^{-4} , per-device batch size 8, gradient accumulation 4, effective batch size 32). Checkpoints are written every $\sim 10\%$ of the run.
6. **Resolve the final checkpoint.** After training completes, the latest `checkpoint-N` folder under `outputs/progressive_{region}/period_{period}/` is selected as the period’s official checkpoint and recorded in the training manifest.

3.6.3 Per-Period Evaluation

Once both the Western and Eastern checkpoints are available for a given period, the pipeline automatically runs the evaluation suite from Section 3.4 against them. `evaluate_bias.py` is invoked with both checkpoints and writes a timestamped JSON file containing per-prompt cross-perplexities, marker counts, TTRs, repetition scores, and KL divergences for all 284 prompts. `batch_chat_test.py` is then invoked once per region to dump raw generations for inspection. By the end of a full pipeline run, each of the 21 time periods has both a pair of checkpoints and a complete evaluation, all under `outputs/progressive_evaluations/period_{period}_evaluation/`.

3.6.4 Bookkeeping and Resume Support

The pipeline writes a top-level `training_manifest.json` that records the timestamp, the resolved Eastern and Western checkpoint paths, and the full training config used for each completed period. This serves two purposes. It is the source of truth for which checkpoint corresponds to which period (later analysis scripts read it instead of guessing from directory names), and it makes the pipeline restartable: if a run is interrupted, `--resume-from <period>` skips ahead to the named period and continues from there without redoing earlier work. Combined with the cumulative-loading design, this means a long pipeline run can be paused, resumed across days, or rerun for a single period in isolation, without the rest of the pipeline knowing or caring.

3.6.5 What the Pipeline Does Not Do

A few things are out of scope. The pipeline does not do hyperparameter search: the values in `DEFAULT_CONFIG` are fixed across all periods and both regions. It does not do early stopping on validation loss; training runs until `max_steps` is reached. There is no post-training, fine-tuning, or RLHF, in line with the scope set out in Section 2.4. Comparing the resulting checkpoints with those from a different training procedure is handled separately in Sections 4.2 and 4.3.

4 Results

This section reports results from three sets of checkpoints and the results we received with various experimentation.

4.1 Fixed-Step Method: Evaluates 2 models - Vanshika

For all the valid data, we ran training on the data using `gpt2training.py` and the evaluation bias were added in the github.

4.1.1 5000 Steps rundown

We trained two GPT-2 models with a fixed step budget of 5,000 steps: the Eastern model trained on 75 files (≈ 46 MB) for approximately 6 hours on the RTX 6000 Ada, and the Western model trained on 119 files (≈ 80 MB) for approximately 2.7 hours on the same hardware. Both models were initialised from random weights and used the same architecture and hyperparameters as the progressive pipeline checkpoints described in Section 3.4.1.

The 5,000-step models immediately showed distinct philosophical character. Eastern outputs gravitated toward Buddhist framing, referencing suttas, the Four Noble Truths, concepts of rebirth and

nirvana, and the register of Pāli canon translations. Western outputs gravitated toward Platonic dialogue structure, metaphysical argument, and the rhetorical register of classical European philosophy. This confirmed that the corpus separation was producing measurably different distributions even at modest step counts.

However, both models generated severely degenerate repetitive output when prompted without decoding constraints—repeating the same phrase or question up to ten or more consecutive times before the generation limit. This is a known failure mode for small models (124M parameters) trained for limited steps without a repetition penalty. The bias signal was present, but generation quality was insufficient for clean evaluation. Table 2 reports the full per-category evaluation for the 5,000-step checkpoints.

Table 2: Bias evaluation results for the 5,000-step fixed checkpoints. W-Rep/E-Rep: repetition score (lower is better); W-TTR/E-TTR: type–token ratio (higher is better); W→E PPL: Western model perplexity on Eastern output; E→W PPL: Eastern model perplexity on Western output; East%: fraction of concept markers from the Eastern tradition.

Category	W-Rep	E-Rep	W-TTR	E-TTR	W→E PPL	E→W PPL	W East%	E East%
Self Identity	0.002	0.001	0.836	0.843	20,494	26,844	0.41	0.62
Purpose Meaning	0.001	0.000	0.812	0.845	26,241	50,242	0.58	0.75
Ethics Morality	0.000	0.000	0.832	0.845	20,569	56,837	0.66	0.58
Reality Existence	0.002	0.001	0.801	0.840	23,467	27,486	0.43	0.39
Knowledge Truth	0.001	0.003	0.825	0.838	18,764	35,286	0.46	0.49
Death Immortality	0.000	0.001	0.824	0.837	21,730	58,653	0.65	0.78
Nature Universe	0.001	0.002	0.812	0.855	17,637	38,901	0.43	0.40
Enlightenment Lib.	0.001	0.001	0.829	0.838	23,817	52,148	0.79	0.71
Free Will Fate	0.000	0.001	0.848	0.838	21,353	36,985	0.31	0.59
Good Evil	0.000	0.001	0.829	0.833	19,459	45,897	0.46	0.54
Society Justice	0.000	0.000	0.829	0.840	15,624	28,696	0.51	0.58
Death Meaning	0.000	0.001	0.841	0.838	24,156	45,503	0.63	0.85
Body Mind	0.001	0.000	0.822	0.836	18,715	50,836	0.53	0.63
Wisdom Truth	0.001	0.001	0.829	0.847	23,851	57,037	0.69	0.52

4.1.2 Scaling to 15,000 Steps- Vanshika

To test whether more pretraining would resolve the generation quality problems, we retrained both models to 15,000 steps. Type–token ratio improved marginally (0.84–0.89 versus 0.80–0.86 at 5,000 steps) and repetition scores remained near zero. However, cross-perplexity increased dramatically: W→E PPL rose from the 15,000–26,000 range to 100,000–285,000, and E→W PPL rose from 26,000–58,000 to 80,000–270,000. Rather than the models becoming more mutually intelligible, they became far less so.

This perplexity explosion is consistent with the overfitting failure mode described in Section ?? . At 15,000 steps, both models began memorising their training distributions rather than learning generalisable patterns, producing outputs that were increasingly foreign to the other model’s probability estimates. The bias signal also became noisier and harder to interpret. Table 3 reports the full 15,000-step evaluation.

These results directly motivated two design decisions in the progressive pipeline. First, the dynamic step budget in `compute_max_steps` (Section ??) scales steps with corpus size rather than fixing them absolutely, avoiding the over-training that occurred here. Second, the epoch cap (`compute_epoch_capped_steps`) limits training to three full passes over each period’s data, which prevents the model from memorising small early corpora as the 5,000- and 15,000-step fixed runs did.

4.1.3 Post-Training Attempt: Supervised Fine-Tuning- Vanshika

With the 5,000-step base models showing clear bias but poor generation quality, we investigated whether Supervised Fine-Tuning (SFT) could reduce repetition while preserving the learned philosophical distributions. We generated approximately 40 instruction–response pairs per model directly from each base model’s own outputs, then fine-tuned each model for three epochs using those pairs. Table 4 compares base model outputs against SFT outputs on the same prompt set.

Table 3: Bias evaluation results for the 15,000-step fixed checkpoints. Column guide identical to Table 2.

Category	W-Rep	E-Rep	W-TTR	E-TTR	W→E PPL	E→W PPL	W East%	E East%
Self Identity	0.000	0.000	0.869	0.875	198,868	129,261	0.44	0.61
Purpose Meaning	0.002	0.001	0.849	0.860	248,243	161,745	0.46	0.53
Ethics Morality	0.001	0.000	0.867	0.850	176,990	128,295	0.45	0.57
Reality Existence	0.000	0.000	0.855	0.842	114,497	100,956	0.28	0.28
Knowledge Truth	0.001	0.000	0.839	0.859	136,609	184,035	0.48	0.32
Death Immortality	0.001	0.000	0.850	0.866	151,301	171,264	0.63	0.50
Nature Universe	0.000	0.000	0.866	0.885	129,914	81,388	0.47	0.31
Enlightenment Lib.	0.000	0.000	0.860	0.852	284,796	227,835	0.74	0.81
Free Will Fate	0.000	0.000	0.855	0.871	150,966	128,840	0.31	0.33
Good Evil	0.000	0.000	0.869	0.866	218,385	131,140	0.41	0.62
Society Justice	0.000	0.000	0.858	0.866	36,106	52,955	0.31	0.16
Death Meaning	0.000	0.000	0.838	0.865	152,566	164,752	0.56	0.59
Body Mind	0.000	0.000	0.861	0.864	169,106	271,249	0.50	0.57
Wisdom Truth	0.000	0.001	0.845	0.845	255,555	260,202	0.73	0.73

Table 4: Base model versus SFT model generation quality metrics.

Metric	Base Model	After SFT
Avg. response length	492 chars	487 chars
Vocab diversity (TTR)	0.4009	0.2903 ↓
Max repetition score	4.9	10.2 ↑
Keyword overlap	0.1122	0.1122 (unchanged)

This result reinforces that post-training can distort or amplify biases rather than reveal them, supporting our decision to focus exclusively on pre-training.: vocabulary diversity fell from 0.4009 to 0.2903, and the maximum repetition score nearly doubled from 4.9 to 10.2. Keyword overlap was unchanged, indicating no meaningful shift in content. The root cause is a self-training collapse: because the SFT training data was generated from the same repetitive base models, fine-tuning reinforced the degeneracy rather than correcting it. SFT data quality is a hard precondition for SFT to help; generating that data from a degenerate base model violates this precondition.

4.1.4 Post-Training Attempt: LoRA + SFT (Considered and Abandoned)

To avoid destroying the base model’s learned philosophical distributions through full-weight SFT, we investigated Low-Rank Adaptation (LoRA) as an alternative. The proposed approach would freeze the base model parameters and train only small rank-decomposed adapter matrices (approximately 1% of total parameters), then merge the adapter back after training. This would have preserved the corpus-learned distributions in the frozen weights while allowing the adapter to correct generation behaviour.

The approach was abandoned for two reasons. First, the fundamental bottleneck was SFT data quality, not the fine-tuning method: LoRA with self-generated repetitive SFT data would produce the same self-training collapse regardless of adapter strategy. Second, using an external model such as GPT-4 to generate higher-quality SFT responses would introduce a bias confound—responses generated from a model trained on internet-scale data carry that model’s distributional biases into the fine-tuning signal, making it impossible to attribute the resulting behaviour to the philosophical corpus alone. Either path risked invalidating the corpus-attribution claim central to this thesis.

4.1.5 Resolution: Inference-Time Repetition Penalty- Vanshika

The repetition problem was ultimately resolved through inference-time decoding constraints rather than post-training. Adding a repetition penalty (`repetition_penalty=1.3`) and an n -gram repetition block (`no_repeat_ngram_size=4`) to the generation configuration eliminated looping without modifying the model’s parameters or distributions. This is the decoding configuration used for all evaluations reported in Sections 4.2 and 4.2.2, and is described in Section 3.4.1.

The approach keeps the evaluation attributable to pre-training: bias measurements reflect what

the pre-training corpus taught the models, not any post-training intervention. The tradeoff is that the models do not produce instruction-following output in a practical sense; their outputs should be read as philosophical continuations in the style of their training corpora rather than as question-answer responses.

4.2 Progressive Pipeline Checkpoints: Manu

The progressive pipeline trains the Western and Eastern GPT-2 models in lockstep across 21 chronological time periods. The two checkpoints reported here are:

- **Western model:** `progressive_west/period_2000/checkpoint-2118`
- **Eastern model:** `progressive_east/period_2000/checkpoint-1299`

Both checkpoints come from the final time period (*period_2000*), so each model has worked through the entire chronological corpus for its tradition, from the pre-Common Era texts up to the 20th century. The step counts (2118 and 1299) come from the trainer’s dynamic step budget, which scales steps with the size of the time period being trained on and caps the maximum number of epochs at 3. Earlier runs without that cap, at 5,000 and 15,000 steps, collapsed into repetitive output; Section 5 returns to that failure mode in more detail.

4.2.1 Why These Checkpoints Matter: Manu

These are the first checkpoints in our experiment that satisfy all three of the following at once:

1. **They generate fluent, non-degenerate text.** Mean type-token ratio (TTR) is **0.821** for the Western model and **0.850** for the Eastern model on 284 generated responses, with a measured repetition score of **0.000** for both. The 5,000- and 15,000-step checkpoints, by comparison, showed clear vocabulary collapse and high *n*-gram repetition in the `visualize_vocab.py` histograms.
2. **They are linguistically distinct from one another.** The cross-perplexity is large in both directions, so each model is finding the other model’s text very improbable, not just stylistically unfamiliar (Section 4.2.2).
3. **They are conceptually biased toward their training tradition.** Concept-marker frequencies show an asymmetric preference for tradition-specific vocabulary (Section 4.2.3).

Earlier runs failed at least one of these conditions, usually the first.

4.2.2 Cross-Perplexity: Manu

Aggregated over all 284 prompts and 14 thematic categories, the Western model assigns a mean perplexity of **13,722** (median 11,658) to text generated by the Eastern model, while the Eastern model assigns a mean perplexity of **2,288** (median 1,901) to text generated by the Western model. Both values are far above what either model assigns to its own outputs, so the two models are clearly producing different distributions of text.

The asymmetry between the two directions is roughly 6×, which is worth explaining. Most Eastern texts in our pipeline are 19th-century or modern English translations (Legge, Griffith, Sujato), and they share a lot of vocabulary with the modern English used in the Western corpus. The Western corpus is dominated by a narrower English register: the KJV, Jowett’s Plato, Augustine, Aquinas, Kant, Hegel. Western-generated text is therefore partly legible to the Eastern model, since it looks like the same kind of English the Eastern model already saw during training. The reverse is not true, and Eastern-generated text remains very improbable to the Western model.

4.2.3 Concept Marker Bias: Manu

Averaged over all generated responses, the marker counts split sharply between the two models:

- The **Western model** produces text where Western markers (*soul, virtue, reason, logos, justice, substance, dialectic*, etc.) account for **77.7%** of all marker hits, against only **11.1%** for Eastern markers.
- The **Eastern model** produces text where Eastern markers (*dharma, karma, nirvana, atman, tao, sunyata, mindfulness*, etc.) account for **52.7%** of marker hits, against **37.4%** for Western markers.

Both models lean toward their own tradition, but the Western model leans much harder. The reason is the same as for the cross-perplexity asymmetry: every Eastern text comes through English translation, and translators routinely use Western philosophical terms as glosses for Eastern concepts. The Eastern model picks up both vocabularies from its training data and uses them roughly in proportion to how often they appear there.

Per-Category Breakdown. The bias signal is not uniform across categories. Table 5 reports Western- and Eastern-marker ratios separately for each of the 14 thematic prompt categories.

Category	W-mod W%	W-mod E%	E-mod W%	E-mod E%
self_identity	0.79	0.11	0.61	0.39
purpose_meaning	0.75	0.15	0.38	0.52
ethics_morality	0.90	0.05	0.33	0.52
reality_existence	0.76	0.14	0.56	0.29
knowledge_truth	0.75	0.15	0.33	0.53
death_immortality	0.80	0.11	0.27	0.64
nature_universe	0.86	0.10	0.50	0.42
enlightenment_liberation	0.88	0.07	0.27	0.68
free_will_fate	0.68	0.17	0.47	0.38
good_evil	0.75	0.05	0.23	0.62
society_justice	0.87	0.08	0.38	0.47
death_meaning	0.79	0.06	0.28	0.57
body_mind	0.59	0.06	0.34	0.66
wisdom_truth	0.71	0.24	0.27	0.72

Table 5: Mean concept-marker ratios per category. “W-mod W%” = fraction of Western markers in the Western model’s output; “E-mod E%” = fraction of Eastern markers in the Eastern model’s output.

The largest gaps show up in categories where Eastern traditions developed their own technical vocabulary that has no one-to-one Western equivalent: *enlightenment_liberation* (E-mod E% = 0.68 vs. W-mod E% = 0.07), *wisdom_truth* (0.72 vs. 0.24), *death_immortality* (0.64 vs. 0.11), *body_mind* (0.66 vs. 0.06), and *good_evil* (0.62 vs. 0.05). Words like *nirvana, moksha, sunyata, anatta, and dukkha* appear in the Eastern model’s outputs and are essentially absent from the Western model’s.

The smallest gaps (*free_will_fate, self_identity*) are in categories where both traditions discuss the same abstract concepts using overlapping English vocabulary, and where the Eastern model’s broader English exposure makes its outputs look more like the Western model’s.

table shared in discord have minor diff in value, double check plx

4.2.4 Distributional Overlap and the Long-Tail Effect: Manu

The full-vocabulary distributional analysis gives a different picture from the marker analysis. Computed over all generated tokens, the two models have a **Bhattacharyya coefficient of 0.8298** (Bhattacharyya distance 0.1866), which the `stats_analysis.py` interpretation scale labels as “high overlap.” Every individual category is also classified as “high overlap” (BC range 0.72–0.81), and the mean symmetric KL divergence on prompt-conditioned distributions is only **9.29%** on a 0–100 scale.

This looks like it should contradict the marker results from Section 4.2.3, but the two metrics are not measuring the same thing.

The Bhattacharyya coefficient is dominated by high-frequency tokens. Both models produce fluent modern English, so most of every output consists of function words and generic vocabulary — *the, and, is, that, we, they, know, see* — and these tokens appear at similar rates in both training corpora. Their sheer frequency dominates the coefficient and pushes it into the high-overlap range no matter how different the models are at the conceptual level. The marker lists do the opposite: they pick out low-frequency, tradition-defining vocabulary in the long tail, which the BC computation effectively ignores.

The vocabulary counts from the same run support this. Across all generated text the Western model uses **1,049** unique words and the Eastern model uses **1,148**, but only **489** of those words are shared. About 60% of each model’s vocabulary is private to that model. It just doesn’t show up in the BC because the shared 40% is high-frequency enough to dominate the average.

We think this is the outcome we want from the experiment. If the BC had been very low, the models would have been diverging into something like incompatible dialects, which usually means overfitting. If the marker bias had been small, the corpus split would have made no real difference. What we see is somewhere in between: both models still write fluent English drawn from a large shared vocabulary core, but each has built up its own technical philosophical vocabulary in the long tail. The marker analysis is what picks up that tail. The BC just confirms that neither model has stopped being an English language model.

4.2.5 Summary of the Progressive Pipeline: Manu

The *period_2000* checkpoints are the most useful ones from the progressive run. They are not the highest-step models we trained, since the higher-step runs collapsed, but they give us a clean separation between the Western and Eastern models on every metric we tracked. TTR and repetition score show that neither model has degraded. Cross-perplexity is large in both directions, and the concept-marker ratios show the kind of tradition-aligned bias we set out to test for. The Bhattacharyya overlap of 0.83 keeps this honest by confirming that both models are still fluent English language models. Section 5 picks up from here and discusses why the longer runs failed.

4.3 Cross-Pipeline Comparison- Vanshika

The fixed-step runs from Section 4.1 and the progressive pipeline from Section 4.2 tell a consistent story when placed side by side, but the story is more nuanced than a simple ranking. Table 6 summarises all three checkpoint sets on a common axis.

Table 6: Summary comparison of all checkpoint sets. Progressive pipeline values are means over all 284 prompts; fixed-step values are ranges across the 14 prompt categories.

Pipeline	Steps	W→E PPL	E→W PPL	Repetition	TTR
Fixed (Section 4.1)	5,000	15,000–26,000	26,000–58,000	0.000–0.003	0.80–0.86
Fixed (Section 4.1.2)	15,000	100,000–285,000	80,000–270,000	0.000–0.002	0.84–0.89
Progressive (Section 4.2)	≈2,000 (final period)	13,722 (mean)	2,288 (mean)	0.000	0.82–0.85

Although the progressive pipeline’s final checkpoint (*period_2000*) trains on the complete corpus, the same texts available to the fixed-step runs—the two pipelines use that data differently. The fixed-step runs load the full corpus as a single flat stream and train until a hard step limit is reached, with no constraint on the number of passes over the data. The progressive pipeline trains 21 cumulative models per region, each on all texts from the oldest period up to the current one, with a dynamic step budget and a three-epoch cap per period (Section 4.2). The *period_2000* checkpoint is therefore the first to have seen the complete dataset, but it arrives at it through a controlled procedure rather than open-ended repetition. The practical consequence is that the final progressive checkpoint trained for only ≈1,299 steps (Eastern) and 2,118 steps (Western) on the full corpus, far fewer than the fixed 5,000-step baseline because the epoch cap prevented additional passes once the data had been covered three times

The practical consequence is that the progressive pipeline’s final checkpoint ran for approximately 1,299 steps (Eastern) and 2,118 steps (Western)—far fewer than the fixed 5,000-step baseline—yet produced cleaner metrics. This confirms that **training duration alone does not determine output**

quality; the epoch cap and dynamic step budget are what prevent the memorisation and repetition failures observed in the fixed-step experiments.

What the progressive pipeline does well. The progressive pipeline’s period-2000 checkpoints are the only ones that simultaneously satisfy all three quality criteria defined in Section 4.2: zero repetition across all 284 prompts, meaningful cross-perplexity in both directions, and a clear concept-marker bias signal. Neither fixed-step checkpoint set satisfies all three at once. The 5,000-step models show good bias signal but suffered generation-time repetition without decoding constraints. The 15,000-step models show near-zero repetition but their cross-perplexity values are so extreme (up to 285,000) that they reflect distribution memorisation rather than meaningful divergence.

An important caveat: the E→W direction. The progressive pipeline’s E→W cross-perplexity of 2,288 (mean) is notably lower than the fixed 5,000-step run’s range of 26,000–58,000. This is worth examining carefully. A lower E→W value means the Eastern model finds Western-generated text less surprising—which could indicate either that the models are less divergent, or that the progressive Eastern model is undertrained relative to the fixed 5,000-step Eastern model and has therefore not developed as strong a philosophical distribution. The progressive pipeline’s final-period checkpoint ran for only $\approx 1,299$ steps on the full cumulative corpus, compared to 5,000 steps in the fixed run. It is plausible that the fixed 5,000-step Eastern model, having seen the same data for more steps, learned a sharper distribution that found Western text more genuinely foreign.

This does not undermine the progressive pipeline’s utility—its primary purpose is to produce 21 temporal snapshots of how bias develops across chronological time periods, not to maximise cross-perplexity on the final checkpoint. But it does mean the final-period progressive checkpoint should not be read as straightforwardly “better” than the fixed 5,000-step checkpoint on divergence metrics. The two pipelines measure somewhat different things.

What the fixed-step runs contribute. The fixed-step runs are not simply failed predecessors to the progressive pipeline—they provide the empirical evidence that motivates the progressive design. Without the 15,000-step results showing PPL explosion, the epoch cap in the progressive pipeline would be an unmotivated design choice. Without the 5,000-step SFT collapse, the decision to resolve repetition via decoding constraints rather than post-training would lack justification. The fixed-step experiments are the empirical grounding for the progressive pipeline’s key decisions.

5 Conclusion and Limitations

This thesis set out to answer a primary question: does the choice of pre-training corpus measurably shape the philosophical vocabulary and conceptual framing of a small language model? A secondary question follows from this investigation: to what extent can such findings help explain whether modern language models exhibit cultural or philosophical bias as a result of their training data?

The results provide a clear answer to the primary question. Two GPT-2 models trained from identical architectures but on disjoint philosophical corpora—one Eastern and one Western—developed measurably different distributions even at modest training scales. Cross-perplexity consistently showed that each model assigns low probability to the other’s outputs, indicating divergence in learned representations. This was further supported by concept-marker analysis, where the Western model produced 77.7% Western markers, while the Eastern model showed a more moderate but consistent preference for Eastern markers (52.7%). This asymmetry is partly explained by the translation-dominated nature of the Eastern corpus.

The strength of the bias signal varied across domains. It was strongest in categories with highly specialized Eastern vocabulary and weakest where both traditions share overlapping conceptual language. This suggests that model bias is shaped not only by data volume but also by the presence of distinct conceptual vocabularies within the training corpus.

Training dynamics further clarified this effect. Lower-step models captured early bias signals but suffered from repetition, while higher-step models showed signs of overfitting, including inflated cross-perplexity and reduced interpretability. Intermediate checkpoints—particularly the 2,000-step model—provided the best balance between coherence and measurable divergence, making them the most reliable for evaluating corpus-induced effects.

In relation to the secondary question, these findings suggest that biases in modern language models are more likely a consequence of their training data than of their architecture. Because both models in this study share identical architectures and differ only in their corpora, the observed differences can be attributed directly to data. This supports the interpretation that real-world language models may inherit and reflect cultural imbalances present in their training distributions.

Three limitations bound these conclusions. First, the 124M-parameter GPT-2 architecture is relatively small, and larger models may exhibit stronger or more nuanced effects. Second, the Eastern corpus consists primarily of English translations, which may introduce stylistic artifacts not present in the original texts. Third, the concept-marker lists were manually curated, and alternative selections could shift quantitative results.

Overall, this work demonstrates that pre-training data alone is sufficient to induce systematic differences in how language models represent philosophical ideas. It highlights the importance of dataset composition in shaping model behaviour and suggests that culturally balanced training data is critical for developing more representative and equitable AI systems.

References

- [BDVJ03] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 2003.
- [GBC16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- [Gol16] Yoav Goldberg. A primer on neural network models for natural language processing. *Journal of Artificial Intelligence Research*, 2016.
- [HG16] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.
- [JM23] Daniel Jurafsky and James H. Martin. *Speech and Language Processing*. 2023. Draft available online.
- [MCCD13] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.
- [RWC⁺19] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Technical Report*, 2019.
- [VSP⁺17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [XYH⁺20] Ruibin Xiong, Yunchang Yang, Di He, Kaiheng Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *International Conference on Machine Learning (ICML)*, pages 10524–10533. PMLR, 2020.